# Project Geavanceerde Computerarchitectuur

Gust Brouckaert
Tim Depickere

14 december 2024

## 1 Introductie

Voor de laatste grote labo-opdracht is het de bedoeling een project te realiseren dat de opgedane technieken en kennis in de praktijk brengt. Het hoofddoel van deze opdracht is om de geleerde concepten op een correcte en effectieve manier toe te passen.

## 2 Opdrachtbeschrijving

Het gekozen project richt zich op Anomaly Detection in Time Series Data. Het doel is om uit een dataset afwijkende waarden (uitschieters) te identificeren. Voor dit project wordt het model getraind in Python. Vervolgens worden de inputvector, gewichten en bias geëxporteerd naar een CSV-bestand. Dit bestand wordt ingelezen in het CUDA-project, waar de benodigde matrixberekeningen efficiënt worden uitgevoerd. Voor er begonnen wordt aan het CUDA project, worden deze berekeningen eerst in een C++ omgevingen uitgetest.

## 3 Dataset

Als dataset hebben we gekozen voor de ECG, electro cardiogram, dataset van Kaggle. Deze dataset bevat 4998 patienten met daarbij horend een 140 kolommen van ECG waardes. De laatste kolom, kolom 141, bevat een bit die voorstelt of deze patient normaal of abnormale waardes heeft.

| | | | | | |
|---|---|---|---|---|---|
| -0.11252183 | -2.8272038 | -3.7738969 | -4.3497511 | -4.376041 | -3.4749863 |
| -1.1008778 | -3.9968398 | -4.2858426 | -4.5065789 | -4.0223767 | -3.2343676 |

Table 1: 5 kolommen per rij als voorbeeld

## 4 Python

Het gekozen model voor anomaly detection is een autoencoder. De encoder bestaat uit vier lineaire lagen, waarbij elke laag wordt gevolgd door een ReLU-activatielaag. De decoder is gelijkaardig opgebouwd, maar bevat aan het einde nog een Sigmoid-laag om de waarden te normaliseren. Deze extra laag was

nodig omdat de data vooraf was geschaald met een MinMaxScaler, wat ervoor zorgt dat alle inputgegevens binnen het bereik [0, 1] vallen.

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim=32):
        super(Autoencoder, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 140),
            nn.ReLU(),
            nn.Linear(140, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, latent_dim),
        )
        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 128),
            nn.ReLU(),
            nn.Linear(128, 140),
            nn.ReLU(),
            nn.Linear(140, input_dim),
            nn.Sigmoid(),  # To match normalized input range
        )
```

Voor de implementatie zijn twee Python-scripts gebruikt. Het eerste script werd gebruikt om het model op te slaan, terwijl het tweede script de gewichten en biases van elke laag extraheerde. Dit tweede script stelde ons in staat om de resultaten van GPU/CPU-berekeningen te controleren zonder telkens een nieuw model te hoeven genereren. Door deze aanpak voorkwamen we dat er bij elke nieuwe sessie andere gewichten en biases werden aangemaakt, wat de reproduceerbaarheid van de resultaten ten goede kwam.

Het model is uitsluitend getraind op niet-anomalie data, zodat afwijkingen in de anomalie data duidelijker naar voren kwamen. Voor het onderscheiden van anomalieën via de GPU werd een drempelwaarde voor de Mean Square Error (MSE) gebruikt. Wanneer de testdataset door het Python-model werd geleid en de MSE tussen het originele en het gereconstrueerde signaal werd geplot (zie figuur 1), zagen we een duidelijke scheiding tussen anomalieën en reguliere data. Dankzij dit duidelijke verschil konden we op basis van de MSE bepalen of een signaal als anomalie of als normaal moest worden beschouwd.

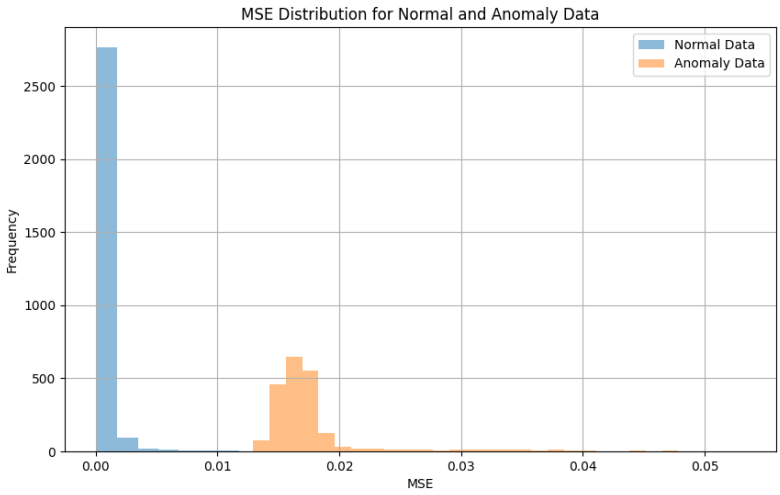



Figure 1: Resultaten: Distributei van MSE

# 5 CPU-Test

Als beginnende test hebben we eerst alles op de CPU getest en berekend. Dit kwam er op neer dat we de verschillende lagen die we gebruikten in ons python model eerst moesten om zetten naar de c++ code. Hiervoor moesten we eerst in de Pythorch documentatie gaan kijken wat deze lagen specifiek doen. De lineaire lagen moesten als volgt berekend worden:

$$\mathbf{y} = \mathbf{x} \cdot \mathbf{A}^T + \mathbf{b}$$

$\mathbf{y}$ : Output vector
$\mathbf{x}$ : Input vector
$\mathbf{A}$ : Gewichten matrix
$\mathbf{b}$ : Bias vector

De ReLu laag:

$$\mathbf{y} = \text{ReLU}(\mathbf{x}) = \max(0, \mathbf{x})$$

$\mathbf{y}$ : Output vector
$\mathbf{x}$ : Input vector

De Sigmoid laag:

$$\mathbf{y} = \frac{1}{1 + e^{-\mathbf{x}}}$$

$\mathbf{y}$ : Output vector
$\mathbf{x}$ : Input vector

De code van deze functies hebben we in bijlage, hoofdstuk 8, toegevoegd. Eenmaal al deze berekeningen gedaan waren hadden we uiteindelijk dezelfde waarde voor onze MSE zoals in onze Pythonomgeving. Dit betekende dat we stap konden overzetten naar het CUDA-programmeren.

# 6 CUDA

Voor de overzet van C++ naar CUDA te zetten was het belangrijkste de 3 functies, forward_pass, relu en sigmoid aan te passen zodat we deze met de gpu konden gebruiken. Ook moesten we onze weigth, bias, input en output matrixes voor de verschillende lagen op de GPU alloceren en de correcte waarden hier inzetten. Aangezien een layer verderwerkt op de output van de vorige layer kon dynamische parallellisatie hier niet worden toegepast.

## 6.1 Thread Divergence en Striding

Aangezien elke operatie exact even lang duurt komt er geen thread divergence binnen een warp zijn doordat bepaalde threads sneller klaar zouden zijn. Aangezien onze input 140 features heeft elke weights matrix 140 kolommen voor elke rij. Bij de matrix transformatie zal 1 thread de volledige multiplicatie doen voor zijn element dus dit zal ook geen thread divergence veroorzaken. Echter is de input size van de autoencode hierdoor ook 140 wat ervoor zal zorgen dat er hier wel thread divergence optreed. De eerste 2 layers van de encoder hebben 140 input elementen waardoor er in 1 warp slecht 12 threads van de 32 nodig zullen zijn. Dit heeft geen grote invloed bij ons aangezien we 192 cores hebben per SM en er dus geen threads staan te wachten die wel nuttig werk zouden kunnen verrichten. Aangezien dit een eigenschap van onze input data is konden we dit ook niet voorkomen.

Er werd ook voor de volledigheid striding toegepast. Dit zodat de code eenvoudig hergebruikt kan worden voor data waar er meer input features zouden zijn. In het huidige geval zullen er nooit meer dan 140 threads nodig zijn dus is striding niet noodzakelijk.

### 6.2 Memory management

Aangezien de weights matrixes en de bias vectors read only zijn kunnen deze gealloceerd worden in de constant memory van de GPU. Echter hebben we slechts 64Kb constant memory dus paste de weigth matrixes niet in de constant memory. De biases werden wel in de constant memory gezet. De

Dit zorgt voor een verbeterde prestatie aangezien het constant memory geen rekening moet houden met synchronisatie. De input en output vectors worden opgeslagen in de global memory aangezien verschillende blocks hiernaar moeten schrijven en lezen.

## 7 Conclusie

Tabel 2 toont het verschil in prestatie voor de CPU via c++ en de GPU via CUDA, we merken een duidelijk verschil op terwijl we met relatief kleine matrixes werken. Hierdoor leiden we af dat onze kernel goed gebruik maakt van de parallellisatie mogelijkheden die de GPU aanbiedt. Voor de waarde van de GPU werd het gemiddelde van 10 functie calls genomen aangezien de eerste keer dat de functie wordt opgeroepen dit trager is. De eerste waarde werd dan ook niet meegerekent bij het gemiddelde. We hebben voor de GPU de performatie gemeten met en zonder het gebruik van Constant memory, we zien hier al een kleine verbetering. Bij input files met minder dan 140 features zal de weights matrix mogelijks ook in de constant memory passen en zal er nog betere preformatie mogelijk zijn.

| | CPU (C++) | GPU (CUDA) | GPU (CUDA) + Constant Memory |
|---|---|---|---|
| Tijd (ms) | 0,3088 | 0,1057 | 0,0979 |

Table 2: Resultaten CPU vs GPU

# 8 Bijlage

```
void forward_pass(const double input[MAX_INPUT_SIZE],
    double weights[MAX_INPUT_SIZE][MAX_OUTPUT_SIZE],
    const double bias[MAX_OUTPUT_SIZE],
    double output[MAX_OUTPUT_SIZE],
    int input_size, int output_size) {

    for (int j = 0; j < output_size; ++j) {
        output[j] = bias[j];

        for (int i = 0; i < input_size; ++i) {
            output[j] += input[i] * weights[j][i];
        }
    }
}

#define MAX(a, b) ((a) > (b) ? (a) : (b))

void relu(double output[MAX_OUTPUT_SIZE], int size){
    for (int i = 0; i < size; ++i) {
        output[i] = MAX(0, output[i]);
    }
}

void sigmoid(double* vector, int size) {
    for (int i = 0; i < size; i++) {
        vector[i] = 1.0 / (1.0 + exp(-vector[i]));
    }
}
```